

ORM

What is an ORM?

ORM = Object Relational Mapper

It lets you talk to a **database using code objects instead of SQL**.

Instead of writing SQL like this: `SELECT`

`* FROM users WHERE id = 1;` You write

something like:

```
const user = await prisma.user.findUnique({  
  where: { id: 1 }  
});
```

So:

Database	Code
Table	Model
Row	Object
Column	Field

users

----- id

name email

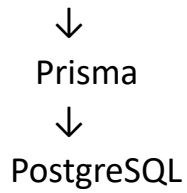
Prisma model

```
model User {  
  id Int @id @default(autoincrement())  
  name String  
  email String  
}
```

Now Prisma converts your code → SQL automatically.

Think of ORM like a **translator between your backend code and the database**.

Node.js code



2. Why ORMs Exist Without

ORM:

You manually write SQL everywhere.

Problems:

- SQL scattered across files
- Harder refactoring
- Harder relations • More boilerplate ORM gives:

- ✓ Type safety
- ✓ Cleaner code
- ✓ Schema management
- ✓ Relations handling
- ✓ Migrations

Prisma Architecture (Important) Prisma

has 3 main parts.

Prisma Schema

Defines database structure.

schema.prisma `model`

`User {`

`id Int @id @default(autoincrement())`

`name String`

```
email String  
}
```

Prisma Client Auto-generated JS

client.

Used in code:

```
const users = await prisma.user.findMany()
```

Prisma Migrate

Handles database migrations.

Example: `npx prisma`

`migrate dev`

Creates SQL migration and updates database.

let's integrate Prisma with PostgreSQL and express with JS (it have problem with TS for now of compilation)

Step 1 — Create a new TypeScript project

```
mkdir hello-prisma
```

```
cd hello-prisma
```

```
npm init -y
```

```
npm install typescript tsx @types/node --save-dev
```

```
npx tsc --init
```

- `typescript` → TypeScript compiler
- `tsx` → run TS files directly
- `@types/node` → Node.js types

Folder Structure

```
project-root/
|
├─ src/
|   ├─ index.ts      <-- Express server
|   ├─ lib/prisma.ts  <-- Prisma client
|   └─ routes/...     <-- optional routes
|
├─ prisma/
|   ├─ schema.prisma
|   └─ migrations/
|
├─ dist/              <-- JS output after tsc
├─ package.json
├─ tsconfig.json
└─ .env
```

Step 2 — Install Dependencies

```
npm install prisma @prisma/client pg @prisma/adapters pg dotenv
npm install --save-dev @types/pg
```

Package roles:

- prisma → CLI (migrate, generate)
- @prisma/client → type-safe DB client
- pg → PostgreSQL driver
- @prisma/adapters → Prisma Postgres adapter
- dotenv → load .env variables
- @types/pg → TypeScript types for pg

Step 3 — Configure TypeScript for ESM

tsconfig.json:

```
{
  "compilerOptions": {
    "target": "ES2022",          // modern JS
    "module": "ESNext",         // ESM
    "moduleResolution": "node",
    "outDir": "dist",           // compiled JS goes here
    "rootDir": "src",           // source code folder
    "strict": true,             // enable strict TS checks
    "esModuleInterop": true,     // allow import express / prisma
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true,       // skip type checking node_modules
    "resolveJsonModule": true,   // allow importing .json files
    "noImplicitAny": true
  },
  "include": ["src/**/*.ts"],    // include all TS files in src
  "exclude": ["node_modules", "dist"] // exclude build folder
}
```

package.json:

```
{
  "type": "module"
}
```

Step 4 — Initialize Prisma ORM with Postgres

```
npx prisma init --db --output ../generated/prisma
```

- Creates prisma/ folder with schema.prisma
- Creates .env for DATABASE_URL
- Creates prisma.config.ts for Prisma 7 config

prisma.config.ts:

```
import "dotenv/config";
import { defineConfig, env } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: { path: "prisma/migrations" },
  datasource: { url: env("DATABASE_URL") },
});
```

Step 5 — Define Your Data Models

prisma/schema.prisma:

Step 6 — Apply Migrations & Generate Client

```
npx prisma migrate dev --name init
npx prisma generate
```

- Creates tables in Postgres
- Generates Prisma client in ../generated/prisma

Step 7 — Instantiate Prisma Client

lib/prisma.ts:

```
import "dotenv/config";
import { PrismaPg } from "@prisma/adapters-pg";
import { PrismaClient } from "../generated/prisma/client";
```

```
const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL
});
const prisma = new PrismaClient({ adapter });

export { prisma };
```

Step 8 — Create Express Server

index.ts:

```
import { prisma } from "../lib/prisma";
```

Step 9 — Run the Server

FOR DEPLOYMENT

Step 1 — Prepare Your Project for Production

1. Make sure your package.json has a **start script**:

```
"scripts": {
  "dev": "tsx index.ts",
  "start": "node --loader tsx/index.ts"
}
```

Render runs npm start by default.

2. Ensure you have **.env variables** set locally (for Render, we'll use the dashboard to set them):

```
DATABASE_URL="postgres://user:password@host:port/dbname"
```

Step 2 — Push Your Code to GitHub

Step 3 — Set Environment Variables in Render // add cloud url not docker one

Step 4 — Create a Render Web Service

1. In Render Dashboard → New → Web Service
2. Connect your GitHub repo
3. Select **Node** as environment
4. Set **Build Command**:
5. `npm install && npx prisma migrate deploy && npx prisma generate`
6. Set **Start Command**:
`npm start`
6. Select **Environment**: Node 18+

Step 7 — Test Your Deployment

The Prisma Change Cycle (Mental Model)

Whenever you **add a table / change a column / add relation**, you repeat this loop:

Edit Schema



Run Migration



Generate Client That's

it.

Prisma Relations

This is how tables connect.

Database thinking:

User



Posts



Comments **Prisma**

Schema Example `model`

```
User {  
  id Int @id @default(autoincrement())  
  name String  
  
  posts Post[]  
}  
  
model Post {  
  id Int @id @default(autoincrement())  
  title String  
  
  authorId Int  
  author User @relation(fields: [authorId], references: [id])  
}
```

Creating Related Data Example:

create **user + post**

```
await prisma.post.create({  
  data: { title: "My First  
Post", author: {  
    connect: { id: 1 }  
  }  
}})
```

Meaning:

create post attach
to user id=1

Select vs Include (VERY IMPORTANT)

This confuses many developers. **include**

→ **fetch relations** Example:

```
const users = await prisma.user.findMany({
  include: {
    posts: true
  }})
```

Result:

User

```
├─ name
├─ id
└─ posts[]
```

Example output:

```
{
  id:1
  name:"Prajwal"
  posts:[
    { id:1, title:"Hello" }
  ]
}
```

select → **choose fields** Example:

```
const users = await prisma.user.findMany({
  select: {
    name: true
  }})
```

Result:

```
[{ name: "Prajwal" }]
```

Interview Cheat select

→ choose columns

include → fetch relations

Transactions

Transactions ensure **all queries succeed or none run**.

Example: Transfer

money deduct

from A add to B

If one fails → rollback.

Prisma Transaction `await`

```
prisma.$transaction([
  prisma.user.update({
    where: { id: 1 },
    data: { balance: { decrement: 100 } }
  }),
  prisma.user.update({ where:
    { id: 2 },
    data: { balance: { increment: 100 } }
  })
])
```

Pagination (Very Common in APIs) Used

when you have **1000+ rows**.

Example API:

GET /posts?page=2

Offset Pagination

```
const posts = await prisma.post.findMany({
  skip: 10,
  take: 10
})
```

Meaning:

skip first 10 take

next 10

SQL equivalent:

LIMIT 10 OFFSET 10

Cursor Pagination (advanced) Better

for large datasets.

```
const posts = await prisma.post.findMany({  
  take: 10, cursor: { id: 20 } })
```

Meaning:

start after id 20 fetch

next 10

Filtering

Filtering = SQL WHERE.

Example: GET

/posts?author=1

Prisma:

```
const posts = await prisma.post.findMany({  
  where: {  
    authorId: 1  
  } })
```

Advanced Filtering `const posts = await`

```
prisma.post.findMany({  
  where: {  
    title: {  
      contains: "Prisma"  
    }  
  } })
```

Result:

posts containing "Prisma"

Multiple Filters `where: {`

```
  authorId: 1,  
  title: {  
    contains: "API"  
  }  
}
```

SQL equivalent:

```
WHERE authorId = 1  
AND title LIKE '%API%'
```